

Monopoly Lite Edition - Análisis, diseño y preparación

1. Reparto de Roles.

- **Responsable de Desarrollo (Lead Developer): Alberto Melero**
 - Responsabilidades: Coordinar la implementación del código en Java, supervisar la arquitectura del proyecto (paquetes y clases) y asegurar que se sigan las buenas prácticas de POO (Programación Orientada a Objetos). Es el encargado de realizar la revisión técnica final antes de integrar cambios en la rama dev.
- **Responsable de Documentación: Alberto Serrano**
 - Responsabilidades: Elaborar y mantener el manual del juego, el diagrama de clases UML y los informes de cada fase. Su objetivo es garantizar que la documentación técnica coincida exactamente con lo que el código hace y que los entregables cumplan con el formato PDF requerido.
- **Responsable de Comunicación y Organización: Javier Cabrera**

Responsabilidades: Gestión del tablero Kanban en GitHub Projects. Se encarga de definir las "Issues" (tareas), asignarlas a los miembros y moverlas entre las columnas (Backlog, In Progress, Done) para que el flujo de trabajo no se detenga. Coordinará las breves reuniones de sincronización del equipo.
- **Programador (Soporte Técnico): Javier García**
 - Responsabilidades: Implementación de funcionalidades específicas y lógica de negocio. Trabjará mano a mano con el Responsable de Desarrollo, participando activamente en la creación de pruebas unitarias y colaborando en la revisión de los Pull Requests de sus compañeros.

2. Bienvenido a la Ciudad Lite

En **Monopoly Lite Edition**, te conviertes en un inversor inmobiliario con un objetivo claro: construir un imperio financiero y sobrevivir a las crisis económicas que llevarán a tus oponentes a la quiebra. Es un juego de astucia, gestión de recursos y, por supuesto, un poco de suerte con los dados.

3. El Gran Objetivo

Para ganar, debes ser el último jugador en pie con dinero en su cuenta. Si el juego se pacta por tiempo o turnos, ganará quien posea la mayor fortuna total, sumando su dinero en efectivo y el valor de todas sus propiedades.

4. Preparación del Tablero

Antes de lanzar el dado, esto es lo que necesitas saber:

- **Capital Inicial:** Cada jugador comienza su aventura con un saldo de \$1500 en el banco.
- **Punto de Partida:** Todos los jugadores colocan su ficha en la casilla 0 (SALIDA).
- **El recorrido:** El tablero es circular y consta de 20 casillas únicas.

5. Cómo se Juega

El juego se desarrolla por turnos. En tu turno, realizarás las siguientes acciones:

1. **Lanzar el dado:** Mueves tu ficha tantas casillas como indique el dado (del 1 al 6).
2. **Actuar según la casilla:** Dependiendo de dónde caigas, podrás comprar, pagar o recibir sorpresas.
3. **Fin del turno:** Una vez resuelta tu acción, el turno pasa al siguiente jugador.

6. Reglas del Mercado Inmobiliario

Compra de Propiedades

Si caes en una casilla que no tiene dueño, ¡puedes comprarla!. Solo tienes que pagar el precio marcado al banco. Si no tienes dinero suficiente, simplemente no podrás adquirirla y el turno termina.

Cobro de Alquileres

Si tienes la mala suerte de caer en una propiedad que ya pertenece a otro jugador, deberás pagarle un alquiler de forma automática.

El Bono de Grupo

¡Atención inversores! Si logras comprar todas las propiedades del mismo color (un grupo completo), el alquiler que cobres en esas casillas se duplicará inmediatamente. ¡Es la clave para ganar rápido!

7. Casillas Especiales y Eventos

- **Casilla de SALIDA (0):** Cada vez que completes una vuelta entera al tablero y pases por aquí, el banco te premiará con \$200.
- **Cárcel (Casilla 5 y 15):** Si caes en la casilla 15 ("Ir a la Cárcel"), tu ficha se moverá directamente a la casilla 5.

Mientras estés arrestado, perderás 2 turnos o podrás pagar una multa para salir antes.

- **Impuestos (Casilla 18):** El estado te pedirá un pago fijo obligatorio que irá directo al banco.
- **Suerte (Casillas 2, 7 y 12):** Aquí todo puede cambiar. Robarás una carta que puede darte dinero o quitártelo.

8. Guía Rápida de Casillas (El Mapa)

Aquí tienes el listado de lo que encontrarás en tu recorrido:

Posición	Nombre	Tipo de Casilla
0	SALIDA	Cobras \$200
1 y 3	Propiedades A1 y A2	Grupo Económico
2, 7, 12	SUERTE	Evento sorpresa
4, 9, 14	Estaciones 1, 2 y 3	Transporte ferroviario
5	CÁRCEL	Visitas o detención
6 y 8	Propiedades B1 y B2	Grupo Medio

10, 11, 13	Propiedades C1, C2 y C3	Grupo Alto
15	IR A LA CÁRCEL	Te envía a la casilla 5
16, 17, 19	Propiedades D1, D2 y D3	Grupo Premium (Las más caras)
18	IMPUESTO	Pago obligatorio

9. Diagrama de Clases (UML)

-  Monopoly Lite.pdf

10. Organización del Proyecto y Control de Versiones

El desarrollo se llevará a cabo bajo un entorno de simulación profesional utilizando las siguientes herramientas:

Flujo de Trabajo en Github

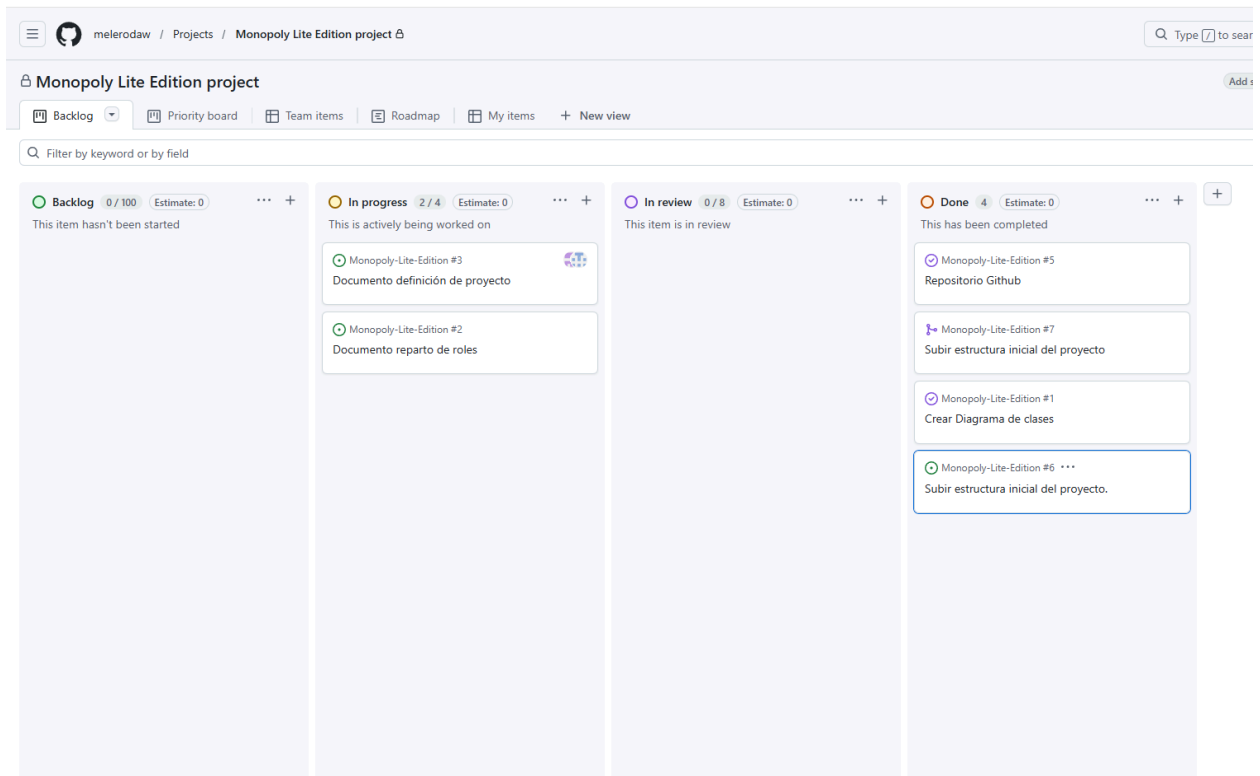
Se ha establecido una jerarquía de ramas para garantizar la estabilidad del código:

- **Rama main:** Contendrá exclusivamente las versiones estables y aprobadas tras finalizar cada fase.
- **Rama dev:** Eje central de desarrollo donde se integran las funcionalidades testeadas.
- **Ramas de Tarea (Features):** Cada miembro trabajará en ramas específicas (ej. feature/clase-jugador) que nacerán de dev. La integración se realizará mediante Pull Requests revisados por al menos un compañero.

Gestión con Tablero Kanban (GitHub Projects)

El seguimiento de tareas se divide en cuatro estados críticos para asegurar la transparencia:

1. **Backlog:** Tareas pendientes de asignación (Diseño UML, Esqueleto Java, Documentación).
2. **In Progress:** Tareas en desarrollo activo con un responsable asignado.
3. **Review:** Código finalizado esperando revisión de Pull Request por parte del Responsable de Desarrollo.
4. **Done:** Funcionalidades aprobadas y mergeadas en la rama dev.



11. Estructura Inicial del Código (Fase I)

Para esta fase, el repositorio incluirá la estructura de archivos `.java` con:

- **Atributos:** Definidos según el diagrama UML.
- **Constructores:** Implementación de constructores por defecto, por parámetros (excluyendo valores aleatorios iniciales) y de copia.
- **Métodos Accesorios:** Getters y Setters necesarios para la interacción entre clases.
- **Visualización:** Implementación del método `toString()` para la monitorización del estado del juego por consola.